

```

# 1. Imports and Dataset Loading
import json
import random
import numpy as np
import nltk
from nltk.stem import PorterStemmer
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from nltk.tokenize import word_tokenize
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input
from tensorflow.keras.optimizers import Adam
import os

# Try to download the resource. The 'try...except' block makes it robust.
try:
    nltk.download('punkt', quiet=True)
    nltk.download('punkt_tab', quiet=True) # Added to fix LookupError
except:
    # If the download fails for any reason, print an error message
    print("Could not download NLTK 'punkt' or 'punkt_tab' resource. Check internet connection.")

# Define sample intents data
intents_data = {
    "intents": [
        {
            "tag": "greeting",
            "patterns": ["Hi", "How are you", "Is anyone there?", "Hello", "Good day"],
            "responses": ["Hello!", "Good to see you again!", "Hi there, how can I help?"]
        },
        {
            "tag": "goodbye",
            "patterns": ["Bye", "See you later", "Goodbye", "Nice chatting to you, bye", "Till next time"],
            "responses": ["See you!", "Have a nice day", "Bye! Come back again soon."]
        },
        {
            "tag": "thanks",
            "patterns": ["Thanks", "Thank you", "That's helpful", "Thank's a lot"],
            "responses": ["Happy to help!", "Any time!", "My pleasure"]
        },
        {
            "tag": "name",
            "patterns": ["What is your name?", "Who are you?", "May I know your name?"],
            "responses": ["I am a chatbot created to assist you.", "You can call me Bot."]
        },
        {
            "tag": "age",
            "patterns": ["How old are you?", "What is your age?"],
            "responses": ["I don't have an age, I'm a program."]
        }
    ]
}

# Create intents.json if it doesn't exist
file_path = 'intents.json'
if not os.path.exists(file_path):
    with open(file_path, 'w') as f:
        json.dump(intents_data, f, indent=4)
    print(f"'{file_path}' created successfully with sample data.")
else:
    print(f"'{file_path}' already exists. Skipping creation.")

# Load the intents data from intents.json
try:
    with open('intents.json', 'r') as file:
        data = json.load(file)
    print("'intents.json' loaded successfully.")
except FileNotFoundError:
    print("Error: 'intents.json' not found. Please ensure it exists.")
    data = {'intents': []}
except json.JSONDecodeError:
    print("Error: Could not decode 'intents.json'. Please check its format.")
    data = {'intents': []}

```

```
'intents.json' created successfully with sample data.
'intents.json' loaded successfully.
```

2. Data Preprocessing, Bag-of-Words, and Label Encoding

```
stemmer = PorterStemmer()

def tokenize_and_stem(text):
    return [stemmer.stem(w.lower()) for w in word_tokenize(text)]

words = []
X = []
y = []

if 'intents' in data and data['intents']:
    for intent in data['intents']:
        tag = intent['tag']

        for pattern in intent['patterns']:
            tokenized = tokenize_and_stem(pattern)

            words.extend(tokenized)

            X.append(tokenized)
            y.append(tag)

words = sorted(list(set(words)))

label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
print("Data preprocessing complete.")
else:
    print("No intents found in 'data'. Skipping preprocessing.")
    # Initialize empty variables to prevent errors if no data was loaded
    words = []
    X = []
    y = []
    y_encoded = np.array([])
    label_encoder = LabelEncoder()

def bag_of_words(tokenized_sentence):
    bag = np.zeros(len(words))

    for w_stemmed in tokenized_sentence:
        if w_stemmed in words:
            bag[words.index(w_stemmed)] = 1
    return bag

# Apply the function to create the bag of words matrix
x_bow = np.array([bag_of_words(sentence) for sentence in X])

# Convert the encoded labels to a numpy array
y_final = np.array(y_encoded)

from sklearn.model_selection import train_test_split

# Split data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x_bow, y_final, test_size=0.25, random_state=42, stratify=y_final)

print(f"Training samples: {len(x_train)}")
print(f"Test samples: {len(x_test)}")
```

```
Data preprocessing complete.
Training samples: 14
Test samples: 5
```

3. Model Building and Training Setup

```
# Ensure necessary Keras components are imported
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input, BatchNormalization # Added BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping

if len(words) > 0 and len(np.unique(y)) > 0:
    # Define the input layer shape based on the vocabulary size
```

```

input_shape = (len(words),)

# Model Definition (Modified for potentially better accuracy)
model = Sequential()
model.add(Input(shape=input_shape))
# First hidden layer: increased neurons, added BatchNormalization
model.add(Dense(256, activation="relu"))
model.add(BatchNormalization())
model.add(Dropout(0.3)) # Adjusted dropout rate

# Second hidden layer: slightly increased neurons
model.add(Dense(128, activation="relu"))
model.add(BatchNormalization())
model.add(Dropout(0.2)) # Adjusted dropout rate

# Third hidden layer (newly added)
model.add(Dense(64, activation="relu"))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Output layer: The number of neurons equals the number of unique tags (classes)
# Softmax activation is used for multi-class probability distribution output
model.add(Dense(len(np.unique(y)), activation="softmax"))

# Model Compilation
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Define a callback for Early Stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True) # Increased patience and added r

model.summary()
print("Model built and compiled successfully with architectural improvements.")
else:
    print("Cannot build model: Not enough unique words or intents after preprocessing.")
    # Create a dummy model to prevent errors in subsequent cells if model cannot be built
    model = None # Or a very basic placeholder if needed later

# 4. Model Training (Add this step if not already present, it's crucial for evaluation)

if model and len(x_train) > 0 and len(y_train) > 0:
    print("Starting model training...")
    # Use x_train and y_train for training, and x_test and y_test for validation
    history = model.fit(x_train, y_train, epochs=200, batch_size=5, verbose=1, callbacks=[early_stopping], validation_data=(x_t
    print("Model training complete.")
else:
    print("Cannot train model: Model not built or data (x_train, y_train) is empty.")

# 5. Evaluation

if model and len(x_test) > 0 and len(y_test) > 0:
    print("Performing model evaluation on test set...")
    y_probs_test = model.predict(x_test)
    y_pred_test = np.argmax(y_probs_test, axis=1)

    acc = accuracy_score(y_test, y_pred_test)
    f1 = f1_score(y_test, y_pred_test, average='weighted')

    print(f"Accuracy on test set: {acc:.4f}")
    print(f"F1-score (weighted) on test set: {f1:.4f}")
    print("\nClassification report on test set:")
    print(classification_report(y_test, y_pred_test, target_names=label_encoder.classes_, zero_division=0))
else:
    print("Cannot evaluate model: Model not built or test data (x_test, y_test) is empty.")

```


Model: "sequential"

```
import matplotlib.pyplot as plt
#accuracy code
```

```
if 'history' in locals() and history is not None:
    # Plot training & validation accuracy values
    plt.figure(figsize=(10, 6))
    plt.plot(history.history['accuracy'])
    plt.plot(history.history['val_accuracy'])
    plt.title('Model accuracy')
    plt.ylabel('Accuracy')
    plt.xlabel('Epoch')
    plt.legend(['Train', 'Validation'], loc='upper left')
    plt.show()
```

```
# Plot training & validation loss values
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

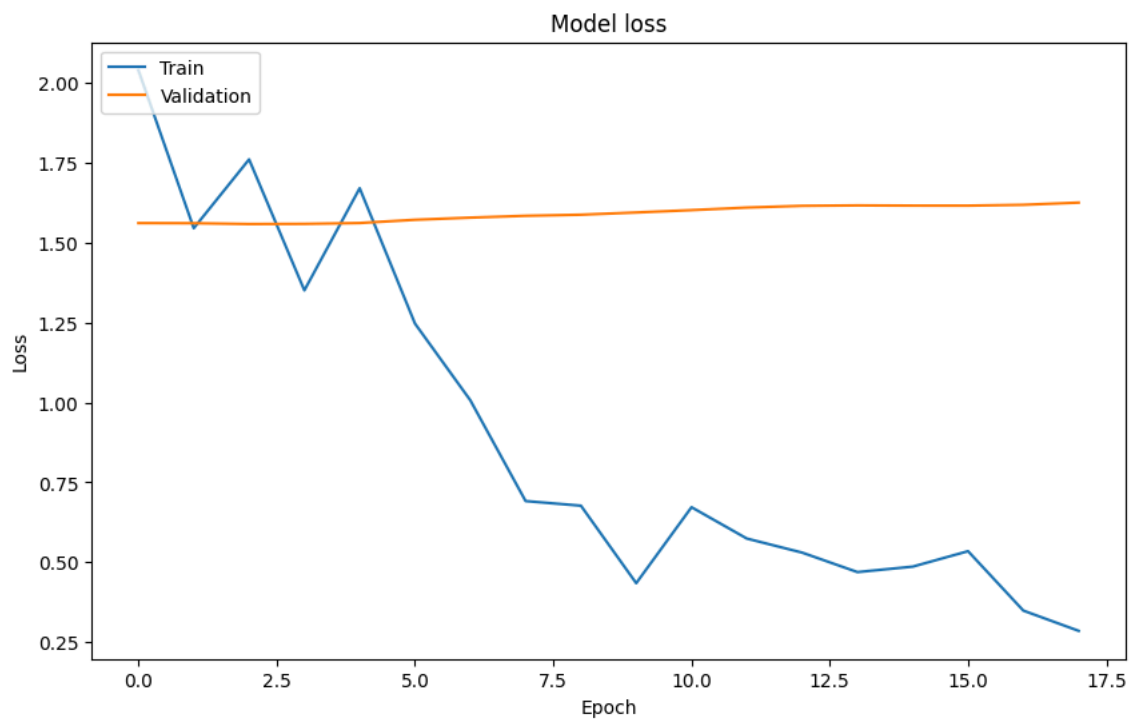
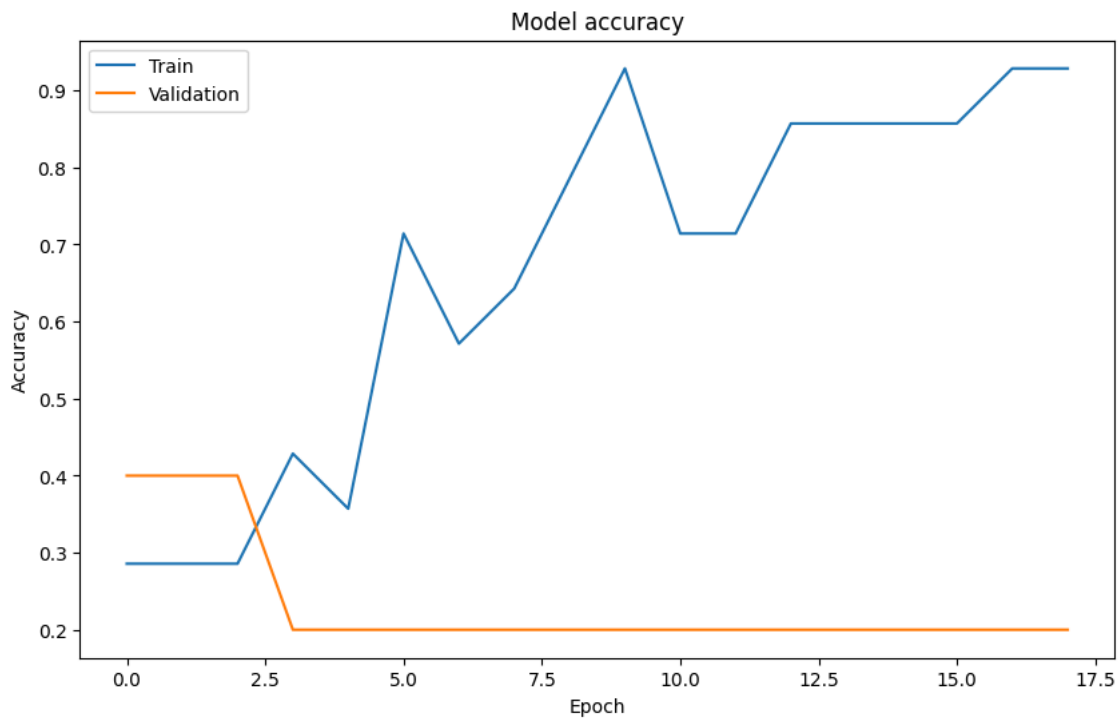
```
else:
    print("Model training history not found. Please ensure the model was trained successfully.")
```

```
new trainable parameters: 650 (150 KB)
Model built and compiled successfully with architectural improvements.
Starting model training...
```

```
Epoch 1/200
3/3 ----- 4s 112ms/step - accuracy: 0.2857 - loss: 2.0399 - val_accuracy: 0.4000 - val_loss: 1.5618
Epoch 2/200
3/3 ----- 0s 23ms/step - accuracy: 0.2857 - loss: 1.5456 - val_accuracy: 0.4000 - val_loss: 1.5613
Epoch 3/200
3/3 ----- 0s 23ms/step - accuracy: 0.2857 - loss: 1.7611 - val_accuracy: 0.4000 - val_loss: 1.5587
Epoch 4/200
3/3 ----- 0s 22ms/step - accuracy: 0.4286 - loss: 1.3511 - val_accuracy: 0.2000 - val_loss: 1.5592
Epoch 5/200
3/3 ----- 0s 23ms/step - accuracy: 0.3571 - loss: 1.6709 - val_accuracy: 0.2000 - val_loss: 1.5619
Epoch 6/200
3/3 ----- 0s 21ms/step - accuracy: 0.7143 - loss: 1.2471 - val_accuracy: 0.2000 - val_loss: 1.5720
Epoch 7/200
3/3 ----- 0s 21ms/step - accuracy: 0.5714 - loss: 1.0071 - val_accuracy: 0.2000 - val_loss: 1.5788
Epoch 8/200
3/3 ----- 0s 23ms/step - accuracy: 0.6429 - loss: 0.6915 - val_accuracy: 0.2000 - val_loss: 1.5845
Epoch 9/200
3/3 ----- 0s 23ms/step - accuracy: 0.7857 - loss: 0.6770 - val_accuracy: 0.2000 - val_loss: 1.5879
Epoch 10/200
3/3 ----- 0s 22ms/step - accuracy: 0.9286 - loss: 0.4344 - val_accuracy: 0.2000 - val_loss: 1.5949
Epoch 11/200
3/3 ----- 0s 21ms/step - accuracy: 0.7143 - loss: 0.6722 - val_accuracy: 0.2000 - val_loss: 1.6023
Epoch 12/200
3/3 ----- 0s 21ms/step - accuracy: 0.7143 - loss: 0.5741 - val_accuracy: 0.2000 - val_loss: 1.6103
Epoch 13/200
3/3 ----- 0s 26ms/step - accuracy: 0.8571 - loss: 0.5301 - val_accuracy: 0.2000 - val_loss: 1.6157
Epoch 14/200
3/3 ----- 0s 22ms/step - accuracy: 0.8571 - loss: 0.4691 - val_accuracy: 0.2000 - val_loss: 1.6171
Epoch 15/200
3/3 ----- 0s 23ms/step - accuracy: 0.8571 - loss: 0.4862 - val_accuracy: 0.2000 - val_loss: 1.6164
Epoch 16/200
3/3 ----- 0s 22ms/step - accuracy: 0.8571 - loss: 0.5345 - val_accuracy: 0.2000 - val_loss: 1.6163
Epoch 17/200
3/3 ----- 0s 23ms/step - accuracy: 0.9286 - loss: 0.3485 - val_accuracy: 0.2000 - val_loss: 1.6189
Epoch 18/200
3/3 ----- 0s 24ms/step - accuracy: 0.9286 - loss: 0.2851 - val_accuracy: 0.2000 - val_loss: 1.6258
Model training complete.
Performing model evaluation on test set...
1/1 ----- 0s 85ms/step
Accuracy on test set: 0.4000
F1-score (weighted) on test set: 0.3333
```

Classification report on test set:

	precision	recall	f1-score	support
age	0.50	1.00	0.67	1
goodbye	0.00	0.00	0.00	1
greeting	0.00	0.00	0.00	1
name	1.00	1.00	1.00	1
thanks	0.00	0.00	0.00	1
accuracy			0.40	5
macro avg	0.30	0.40	0.33	5



3. Model Building and Training Setup

```
# Ensure necessary Keras components are imported
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Input, BatchNormalization # Added BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping

if len(words) > 0 and len(np.unique(y)) > 0:
    # Define the input layer shape based on the vocabulary size
    input_shape = (len(words),)

    # Model Definition (Modified for potentially better accuracy)
    model = Sequential()
    model.add(Input(shape=input_shape))
    # First hidden layer: increased neurons, added BatchNormalization
    model.add(Dense(256, activation="relu"))
    model.add(BatchNormalization())
```

```

model.add(Dropout(0.3)) # Adjusted dropout rate

# Second hidden layer: slightly increased neurons
model.add(Dense(128, activation="relu"))
model.add(BatchNormalization())
model.add(Dropout(0.2)) # Adjusted dropout rate

# Third hidden layer (newly added)
model.add(Dense(64, activation="relu"))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Output layer: The number of neurons equals the number of unique tags (classes)
# Softmax activation is used for multi-class probability distribution output
model.add(Dense(len(np.unique(y)), activation="softmax"))

# Model Compilation
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Define a callback for Early Stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True) # Increased patience and added r

model.summary()
print("Model built and compiled successfully with architectural improvements.")
else:
    print("Cannot build model: Not enough unique words or intents after preprocessing.")
    # Create a dummy model to prevent errors in subsequent cells if model cannot be built
    model = None # Or a very basic placeholder if needed later

# 4. Model Training (Add this step if not already present, it's crucial for evaluation)

if model and len(x_train) > 0 and len(y_train) > 0:
    print("Starting model training...")
    # Use x_train and y_train for training, and x_test and y_test for validation
    history = model.fit(x_train, y_train, epochs=200, batch_size=5, verbose=1, callbacks=[early_stopping], validation_data=(x_t
    print("Model training complete.")
else:
    print("Cannot train model: Model not built or data (x_train, y_train) is empty.")

```



```
# 5. Evaluation

if model and len(x_test) > 0 and len(y_test) > 0:
    print("Performing model evaluation on test set...")
    y_probs_test = model.predict(x_test)
    y_pred_test = np.argmax(y_probs_test, axis=1)

    acc = accuracy_score(y_test, y_pred_test)
    f1 = f1_score(y_test, y_pred_test, average='weighted')

    print(f"Accuracy on test set: {acc:.4f}")
    print(f"F1-score (weighted) on test set: {f1:.4f}")
    print("\nClassification report on test set:")
    print(classification_report(y_test, y_pred_test, target_names=label_encoder.classes_, zero_division=0))
else:
    print("Cannot evaluate model: Model not built or test data (x_test, y_test) is empty.")
```

Performing model evaluation on test set: (None, 64)	256
1/BatchNormalization: 0s 80ms/step	
Accuracy on test set: 0.2000	
F1-score (weighted) on test set: 0.0670	0
Classification report on test set: (None, 5)	325
precision recall f1-score support	